

From Persona Simulation to Data-Driven Prediction: Rethinking LLM-Based Social Data Generation

1 The Problem: SSDataBench and the Failure of Persona Simulation

Xie et al. (2025) introduce SSDataBench, a benchmark evaluating whether LLM-generated social science data reproduce population-level statistical patterns observed in real survey data. The benchmark tests five types of statistical patterns (univariate distributions, bivariate associations, multivariate predictions, life event sequence distributions, and life event sequence associations) across seven major survey datasets and 15 LLMs.

The results are concerning. Pass rates are low across all models, typically below 0.5. Univariate distributions and life event sequences show near-zero pass rates. Most importantly, performance does not improve with model capacity: GPT-5 performs no better than GPT-3.5-Turbo on these tests.

2 My Reading of the Problem

I believe this failure is not a deficiency of LLMs but a natural and predictable consequence of the evaluation paradigm itself.

An LLM is, in effect, an average person. Its training objective (predicting the most likely next token) means that for any given prompt, it produces the most typical response. When you ask it to role-play a specific individual conditioned on demographic variables, it generates what it considers the most plausible life for that profile. This is fast thinking: pattern matching against internalized priors.

Now consider what SSDataBench does: it asks this average person to generate 1,000 individuals, one at a time, and then checks whether the aggregate distribution of those 1,000 cases matches reality. But by construction, 1,000 independent acts of "guessing the most typical answer" will converge toward a narrow set of modal values. The resulting distribution collapses to a Dirac distribution. Associations become exaggerated because the model defaults to stereotypical relationships.

This is exactly what the paper observes, and it would happen to any case-wise predictor optimized for typical accuracy. The problem is not that LLMs lack knowledge about society. The problem is that you cannot recover a population distribution by repeatedly asking a single agent to guess individual cases.

The analogy is straightforward: if you asked a single well-informed human to guess the incomes of 1,000 Americans one by one, conditioned on their demographics, the resulting distribution would also be too narrow and too stereotyped. No individual, no matter how knowledgeable, can mentally simulate population-level heterogeneity through case-by-case intuition. The only way to know what the real distribution looks like is to look at data.

The same applies to LLMs. The solution is not to make the model a better guesser. It is to give the model data and let it reason from data.

The reframing is: input is data, output is also data. Specifically, the data we do not yet have. The model's job is to discover patterns in observed data and use those patterns to infer what the unobserved data should look like, including its full distributional properties.

3 Approach 1: The MRF Approach

One way to operationalize this idea is through a Markov Random Field (MRF) framework.

The core idea: train a large MRF on all available survey data, where the model learns the joint distribution over social variables. The output of the model is not individual predictions but parameters in a functional space, namely the parameters of conditional distributions that characterize how variables relate to each other across the population. Given any set of conditioning variables, the trained MRF can output the distributional parameters (e.g., the mean, variance, and shape of income given education, race, gender, and age) rather than a single point estimate.

The strength of this approach is that it directly encodes the statistical structure of the observed data. All the information contained in existing surveys (marginal distributions, pairwise associations, higher-order dependencies) is injected into the model's parameters through training. Generation from this model would, in principle, preserve distributional fidelity because the model explicitly represents distributions rather than typical values.

But MRF approach has several limitations:

Inflexible structure. The MRF requires the variable set and dependency graph to be defined at training time. Adding a new variable (e.g., a survey question that was not in the training data) requires restructuring and retraining the model. Social science research constantly encounters new variables and new measurement instruments; a fixed-structure model cannot accommodate this.

Fixed input-output schema. The model can only condition on variables it was trained to treat as inputs and can only predict variables it was trained to treat as outputs. If a researcher wants to condition on a different set of variables or predict a variable that was not in the original training set, the model cannot help. Real research questions do not come pre-formatted to match a model's input-output schema.

Limited transferability. An MRF trained on U.S. survey data (e.g., NLSY) cannot be applied to Chinese survey data (e.g., CFPS) without retraining. The learned parameters encode the statistical regularities of a specific population, not generalizable knowledge about social mechanisms. Each new dataset or national context requires a new model.

Scale of engineering effort. Social science data are heterogeneous, mixing continuous, ordinal, nominal, sequential, and time-varying variables. Designing an MRF architecture that handles this full variable space is a significant engineering challenge, and the resulting model serves only the specific datasets it was trained on.

In short, the MRF approach correctly identifies that the solution requires working with data rather than intuition, but it operationalizes this insight through a rigid, special-purpose model that lacks the flexibility real research demands.

4 Approach 2: Data Analyst Agent

I propose an alternative that preserves the core insight (predictions must be grounded in data, not in LLM intuition) but operationalizes it through an agent framework.

Core idea: data-driven prediction via an autonomous agent. Instead of training a dedicated model on survey data, we use an LLM agent as a data analyst. Given a dataset and a prediction target, the agent autonomously explores the data, selects appropriate modeling strategies, writes and executes code to fit models, constructs its own evaluation metrics, iterates across multiple modeling approaches, and delivers predictions that are calibrated against the empirical data.

Workflow

1. **Problem formulation.** The agent receives the dataset and the prediction task (e.g., “given these background variables, predict the distribution of income for this population”).
2. **Data exploration.** The agent writes code to compute descriptive statistics, visualize distributions, identify subgroup structures, examine conditional relationships, and map the overall statistical landscape. This is where the agent builds its understanding of the data, not from prior knowledge alone, but from empirical observation.
3. **Modeling strategy selection.** Based on its exploration, the agent decides what modeling approach to use. This is where LLM intelligence is most valuable: the agent draws on its knowledge of statistical methods, social science theory, and the specific patterns it discovered in the data to choose an appropriate model. Different datasets, different variables, and different prediction tasks may call for different models (mixture models, copulas, Bayesian networks, conditional density estimation, or combinations thereof). The agent decides.
4. **Implementation and calibration.** The agent writes code to implement its chosen model, fits it to the data, and evaluates goodness-of-fit. If the model fails to capture key distributional properties, the agent diagnoses the problem and revises its approach.
5. **Validation.** The agent constructs its own evaluation metrics (potentially including the same statistical tests used in SSDataBench) and validates its predictions against held-out data or through cross-validation. It iterates until validation criteria are met.
6. **Output.** The final output is not a set of individually generated cases but a calibrated predictive model (expressed as executable code) that can generate data preserving population-level statistical properties.

Why This Is Better Than Traditional Approaches

- **Adaptive.** The model structure is not fixed in advance. The agent selects and constructs the appropriate model for each dataset and each task. New variables, new datasets, and new research questions are handled without retraining.
- **Combines two information sources.** The agent uses semantic knowledge (understanding of social mechanisms, statistical methodology) to guide model selection, and uses empirical data to calibrate parameters. In data-sparse regions, semantic knowledge provides useful priors. In data-rich regions, empirical calibration dominates. The MRF has only the latter.

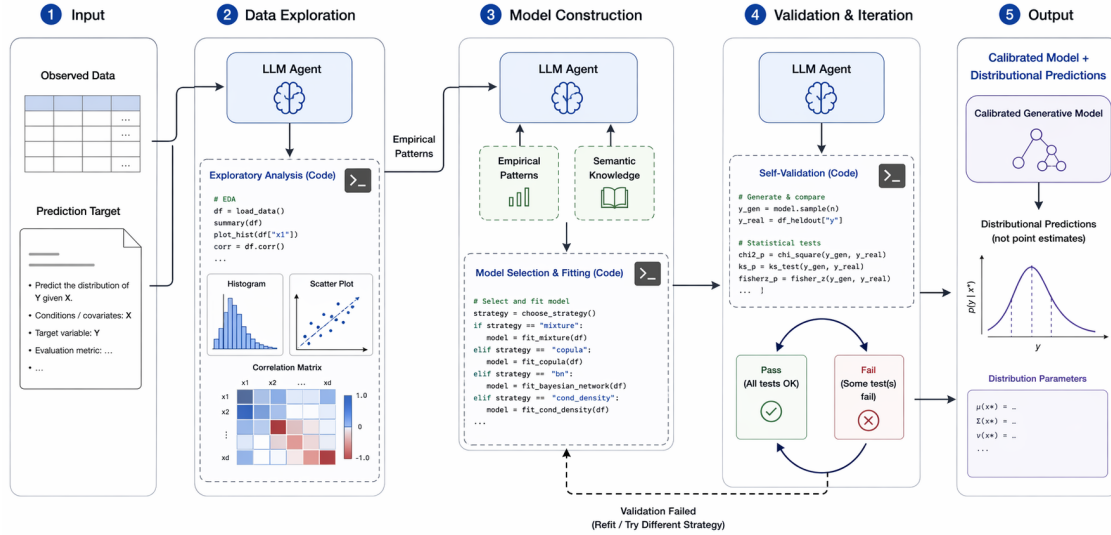


Figure 1: Data Analyst Agent Workflow

- **Transferable.** The agent’s core competence (data exploration, model selection, calibration, validation) is general. It works on NLSY, CFPS, Add Health, or any new dataset without modification.
- **Transparent.** Every step of the agent’s reasoning is recorded as executable code. The modeling choices, the calibration results, and the validation outcomes are all inspectable and reproducible.

Why This Is Better Than Direct LLM Generation

- **Grounded in data.** Predictions are derived from empirical patterns, not from LLM intuition. Distributional properties are preserved because the agent explicitly models distributions.
- **Self-correcting.** The agent validates its own output and iterates. Direct generation has no such feedback loop.
- **Addresses the root cause.** The distributional collapse in SSDataBench occurs because LLMs generate case-by-case via fast thinking. The agent approach replaces this with systematic, data-driven slow thinking.

5 What This Work Would Demonstrate

Using SSDataBench as the evaluation testbed, this work would show:

1. An LLM agent operating as a data analyst (exploring data, building models, and self-validating through code) achieves substantially higher population-level statistical realism than direct LLM generation across all five statistical pattern types.
2. This improvement is not because the agent uses a better generation algorithm, but because it uses a fundamentally different paradigm: data-driven modeling rather than intuition-based simulation.

3. LLMs' value in social data tasks lies not in their ability to simulate individual humans, but in their ability to reason about data, select appropriate methods, and construct calibrated models. In other words, to function as data scientists.